

AI SCIENTIST

FASE 2 | AULA 02 -

INTEGRAÇÃO DE DADOS EM TEMPO REAL (KAFKA)

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SUMÁRIO

O QUE VEM POR AÍ? 3

HANDS ON 4

SAIBA MAIS 7

MERCADO, CASES E TENDÊNCIAS 22

O QUE VOCÊ VIU NESTA AULA? 23

REFERÊNCIAS 25

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VEM POR AÍ?

Já apresentamos os fundamentos dos pipelines ETL e ELT, responsáveis pela movimentação e preparação de dados em arquiteturas analíticas tradicionais. Embora esses pipelines em lote sejam adequados para grande parte dos cenários analíticos, o avanço de aplicações digitais, sistemas orientados a eventos e modelos de Inteligência Artificial que exigem baixa latência impulsionou a adoção de arquiteturas de integração de dados em tempo real.

Essa necessidade impulsionou o surgimento de novas soluções e padrões de arquitetura. Técnicas como Captura de Dados de Mudança (CDC, Change Data Capture) têm ganhado destaque. O CDC permite detectar e replicar alterações ocorridas em bancos de dados transacionais em tempo real, minimizando o atraso e o impacto nas bases de origem. Em vez de extrair periodicamente todo o conjunto de dados, o CDC captura apenas as mudanças (inserções, atualizações, deleções) conforme ocorrem e as encaminha imediatamente para outros sistemas ou plataformas de streaming. Isso viabiliza uma atualização contínua de data warehouses, data lakes e dashboards analíticos com praticamente zero atraso, contrastando com as janelas de horas ou dias dos ETLs tradicionais.

Nesse contexto, o Apache Kafka consolidou-se como uma das principais plataformas de streaming distribuído, permitindo a ingestão, o transporte e o processamento contínuo de grandes volumes de dados.

Em resumo, integrar dados em tempo real significa quebrar silos de dados e reduzir a defasagem entre a geração e o uso da informação, habilitando organizações a serem mais proativas.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

HANDS ON

Laboratório Conceitual: Planejando um Pipeline em Tempo Real

Elaborar o design de um pipeline de integração de dados em tempo real utilizando Kafka e seus componentes, para atender a um cenário de negócio proposto.

Contexto do Problema

Imagine que você trabalha em uma empresa de e-commerce. O time quer passar de relatórios batch diários para análises em tempo real das vendas e do comportamento dos usuários no site. Atualmente, os dados de vendas ficam em um banco de dados relacional PostgreSQL do sistema de pedidos, e os logs de navegação dos usuários são gravados em arquivos de log em um servidor web. O objetivo é integrar esses dados em tempo real em um pipeline que alimente tanto um painel de BI streaming (para a equipe de negócio acompanhar vendas e tendências ao minuto) quanto um sistema de detecção de fraude que deve ser acionado imediatamente a cada transação suspeita.

Siga os passos conceituais abaixo:

1. **Identificação das Fontes de Dados e Eventos:** liste quais eventos/dados específicos precisam ser integrados em tempo real. Provavelmente teremos eventos de **Nova Venda** (contendo detalhes do pedido, valor, cliente, etc.) provenientes do banco de pedidos, e eventos de **Navegação** (page views, cliques) provenientes dos logs do site. Detalhe cada evento e sua origem.
2. **Definição dos Tópicos Kafka:** para cada tipo de evento identificado, defina um tópico no Kafka. Por exemplo, poderíamos ter um tópico vendas para eventos de venda e um tópico web-logs para eventos de navegação. Considere se será necessário particionar esses tópicos e qual chave de partição seria adequada (e.g., usar ID do pedido como chave para garantir que eventos do mesmo pedido caibam na mesma partição e ordem).
3. **Ingestão em Tempo Real (Fonte -> Kafka):** determine como os dados sairão das fontes e chegarão ao Kafka. Aqui, sugira qual **conector do Kafka Connect** poderia ser usado ou qual estratégia. Por exemplo: para o banco PostgreSQL, recomendar o uso de **Kafka Connect + Debezium (CDC)** para capturar inserções/atualizações na tabela de pedidos quase em tempo real.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Para os logs de navegação, poderíamos usar um conector *Source* que lê arquivos de log (existe, por exemplo, o **FileSourceConnector** padrão do Kafka Connect) ou adaptar a aplicação web para publicar mensagens diretamente no Kafka (via Producer API) assim que ocorrem determinadas ações do usuário. Justifique a escolha de cada método de ingestão.

4. **Processamento em Streaming (opcional):** avalie se é necessário processar ou transformar os eventos em trânsito antes do consumo final. No nosso cenário, dois exemplos: (a) Para detecção de fraude, talvez queiramos agregar transações por usuário em uma janela de tempo para identificar atividades suspeitas – isso poderia ser feito com um aplicativo **Kafka Streams** ou uma consulta **ksqlDB** que mantenha uma contagem de compras por usuário nos últimos X minutos, gerando um evento de alerta se ultrapassar um limite. (b) Para o painel de BI, possivelmente não é preciso muita transformação, apenas transmissão dos eventos brutos para serem consumidos pela ferramenta de visualização, mas talvez queira-se usar Streams para unir dados de vendas com dados de cliente (que podem vir de outra fonte) antes de enviar ao dashboard. Se achar necessário, desenhe essas etapas de processamento indicando que tópico(s) de saída serão gerados.
5. **Entrega para Sistemas Destino (Kafka -> Sink):** agora, defina como os dados saem do Kafka para chegar aos consumidores finais. Temos dois consumidores principais no cenário: o sistema de BI em tempo real e o sistema de fraude. Suponha que o dashboard de BI consuma dados através de um banco de dados analítico ou uma ferramenta como Elasticsearch/Kibana. Você poderia sugerir usar um **Sink Connector** do Kafka Connect que pega os eventos do tópico de vendas e insere em um Elasticsearch em nuvem, permitindo visualização imediata. Já para o sistema de fraude, caso ele seja um microserviço interno, ele próprio pode atuar como consumidor Kafka (lendo do tópico de vendas ou do tópico de alertas gerados pelo Streams). Descreva esses destinos e como ocorrerá a integração (conector sink, consumer API, etc.).
6. **Considerações de Escalabilidade e Falhas:** inclua no seu plano aspectos de robustez. Por exemplo: configurar o cluster Kafka com pelo menos 3 brokers e fator de replicação de partições = 3 para tolerar falha de um nó sem perder

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

dados. Mencionar a necessidade de monitorar o lag dos consumidores (o painel de BI e o serviço de fraude) para garantir que estão conseguindo acompanhar o fluxo em tempo real. E prever como escalar: se o volume de eventos web crescer muito, talvez aumentar o número de partições do tópico web-logs e escalar horizontalmente os consumidores do painel que processam esses logs.

7. **Diagrama:** por fim, esboce um diagrama mostrando a arquitetura proposta.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

SAIBA MAIS

Comparativo Arquitetural: Lambda vs. Kappa

Antes de aprofundar o uso do Apache Kafka, é importante contextualizarmos sua adoção na evolução das arquiteturas de Big Data. Essa evolução pode ser compreendida pela transição da Arquitetura Lambda para a Arquitetura Kappa.

A Arquitetura Lambda surgiu como resposta à alta latência do processamento em lote, combinando duas camadas paralelas: uma camada batch, responsável pelo processamento completo e preciso do histórico de dados, e uma camada speed, voltada ao processamento rápido de dados recentes. Embora eficaz, esse modelo introduziu elevada complexidade operacional, pois exigia a duplicação da lógica de negócio e de engenharia de features em duas bases de código distintas.

A Arquitetura Kappa propõe a simplificação desse cenário ao unificar o processamento em torno do streaming como fonte primária de dados, reduzindo redundâncias e favorecendo pipelines mais consistentes e fáceis de manter — contexto no qual plataformas como o Kafka se destacam.

Apache Kafka e o Streaming de Eventos

O Apache Kafka é uma plataforma distribuída de streaming de eventos, originalmente desenvolvida no LinkedIn e posteriormente open source, projetada para alta escalabilidade, baixa latência e tolerância a falhas. Conforme descrito pela AWS (2024) e pela Google Cloud (2024), o Kafka opera como um log distribuído, no qual eventos são organizados em tópicos e persistidos de forma ordenada.

Em uma arquitetura Kafka, produtores publicam eventos em tópicos, enquanto consumidores os leem de maneira independente, possibilitando múltiplos usos simultâneos do mesmo dado. Para pipelines de dados e Inteligência Artificial, isso permite que um único evento alimente dashboards, pipelines analíticos, Feature Stores e modelos de Machine Learning em paralelo, sem acoplamento direto entre sistemas. Resumidamente, podemos definir o Kafka como um sistema de mensageria publish-subscribe, o sistema de mensageria nada mais é do que uma forma de comunicação assíncrona entre duas ou mais aplicações.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Um cluster Kafka é composto por brokers responsáveis por armazenar e gerenciar os registros publicados. Os dados são categorizados em tópicos, que podem ser particionados para distribuir carga e permitir paralelismo de processamento. Essa arquitetura desacoplada garante alta disponibilidade, pois produtores e consumidores não precisam estar ativos simultaneamente, e resiliência, uma vez que os eventos permanecem armazenados conforme a política de retenção e podem ser replicados entre brokers para tolerância a falhas.

O Kafka alcança altas taxas de transferência com baixa latência ao utilizar escrita e leitura sequencial em disco e ao permitir que consumidores avancem no log no próprio ritmo, inclusive reprocessando eventos históricos quando necessário. Além disso, o escalonamento horizontal por meio da adição de brokers e partições possibilita lidar com volumes crescentes de dados sem degradação significativa de desempenho.

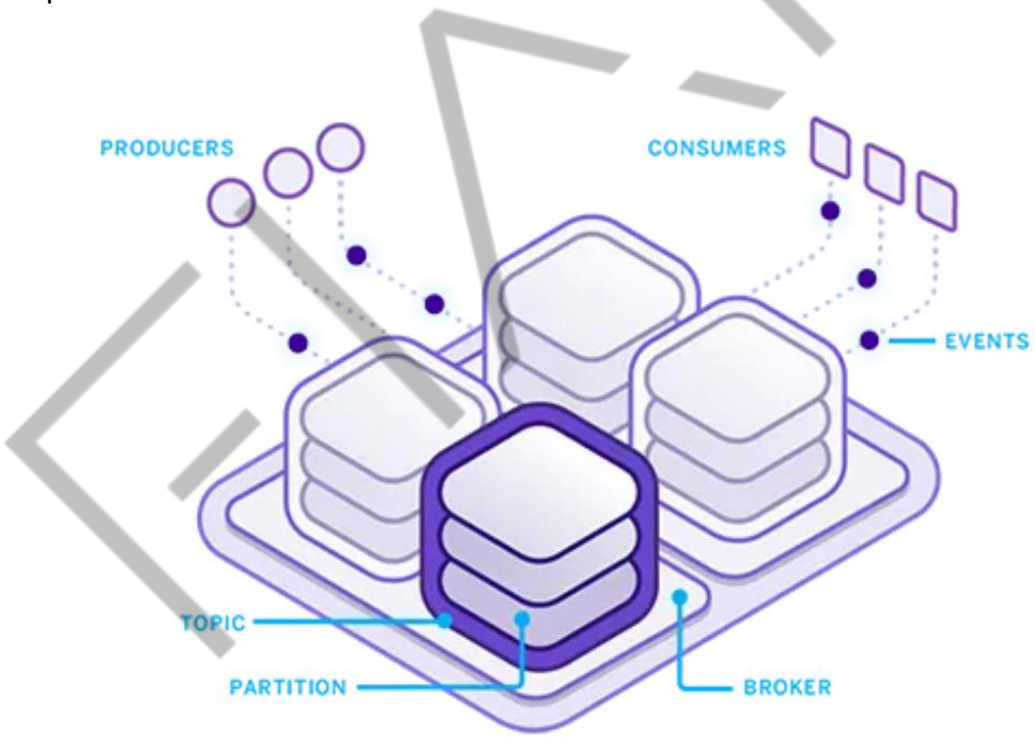


Figura 1 - Representação visual dos fundamentos do Kafka
Fonte: Descomplicando o Apache Kafka - Felipe Lopes (2021)

Diferentemente de sistemas tradicionais de filas de mensagens, nos quais os dados são removidos após o consumo, o Kafka mantém os eventos disponíveis no log por um período configurável, preservando a ordenação dentro de cada partição. Esse

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

modelo o posiciona como um log de eventos distribuído, adequado para padrões como event sourcing, alimentação de data lakes e sincronização de sistemas heterogêneos.

Na prática, o Kafka é amplamente adotado em casos de uso como transmissão de transações financeiras em tempo real, detecção de fraudes, integração entre microsserviços, coleta de telemetria de IoT e ingestão contínua de logs de aplicações. Nos próximos tópicos, serão explorados componentes complementares do ecossistema Kafka, como Kafka Connect e Kafka Streams, que ampliam suas capacidades de integração e processamento de dados em tempo real.

Kafka Connect e Integração com Sistemas Externos

O Kafka Connect é um componente nativo do Apache Kafka voltado à integração padronizada entre sistemas externos e o cluster Kafka. Ele fornece um framework para criação e execução de conectores reutilizáveis, responsáveis por mover dados de forma confiável entre fontes e destinos, eliminando a necessidade de desenvolver aplicações customizadas para cada integração.

Por meio de conectores configuráveis, é possível ingerir dados de sistemas como bancos relacionais, diretórios de arquivos ou APIs e publicá-los em tópicos Kafka, bem como consumir dados desses tópicos e gravá-los em sistemas de destino, como mecanismos de busca, data lakes ou bancos analíticos. Essa abordagem reduz código operacional, padroniza integrações e facilita a manutenção de pipelines em ambientes corporativos.

O Kafka Connect é executado por processos separados Java chamados *workers*, que podem operar de forma distribuída, oferecendo tolerância a falhas e escalabilidade horizontal. O próprio framework gerencia aspectos como offsets de leitura, balanceamento de tarefas e recuperação em caso de falhas, simplificando a operação. Dependendo do cenário, o Connect pode ser executado em modo standalone para testes ou em modo distribuído para ambientes de produção, mantendo o mesmo modelo conceitual de funcionamento.

Cada conector do Kafka Connect atua ou como Source (fonte) ou Sink (coletor). Source connectors se conectam a sistemas de origem (por exemplo, uma tabela de banco, um diretório de arquivos, uma API) e enviam os dados desses sistemas para

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

tópicos no Kafka. Já os Sink connectors fazem o inverso: lêem dados de tópicos Kafka e inserem em sistemas de destino (um banco de dados, um sistema de arquivos, etc.).

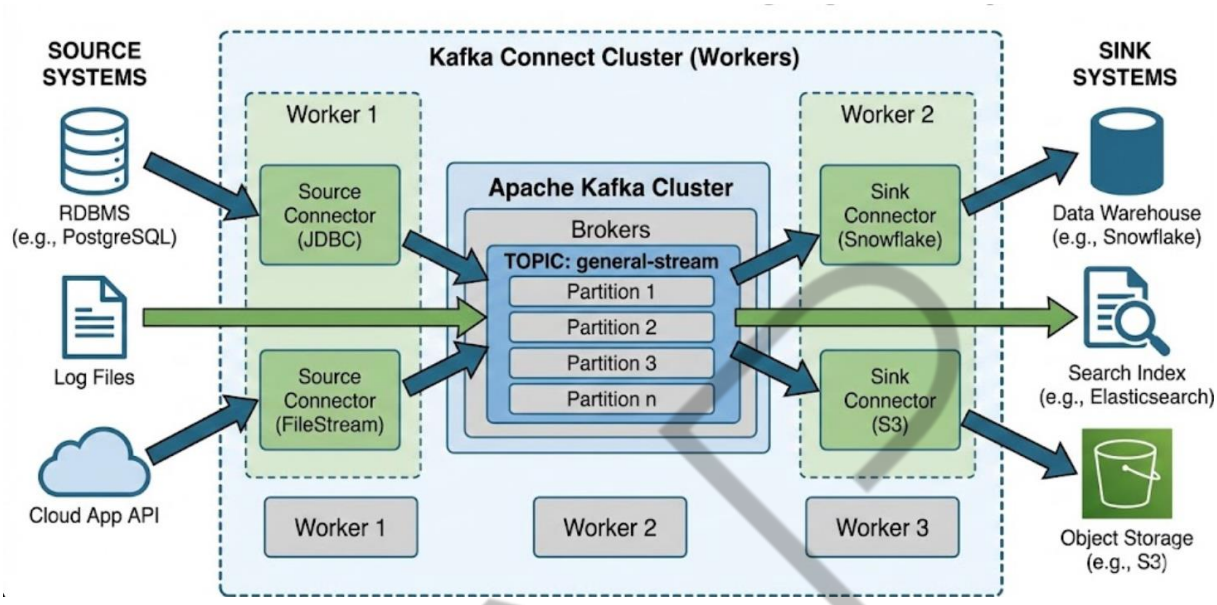


Figura 2 - Exemplificação do funcionamento Kafka Connect
Fonte: Elaborado pelo autor utilizando a ferramenta Nano Banana (2026)

Kafka Streams e Processamento de Dados em Fluxo

Enquanto o Kafka Connect é responsável pela movimentação de dados entre sistemas externos e o cluster Kafka, o Kafka Streams é voltado ao processamento de dados em fluxo (*stream processing*) diretamente no contexto da própria plataforma Kafka. O Kafka Streams é uma biblioteca *client-side* que permite construir aplicações em Java ou Scala capazes de consumir, processar e produzir eventos em tempo real, sem a necessidade de um cluster de processamento separado. Ele disponibiliza uma API de alto nível para transformar *streams* de entrada em *streams* de saída, aplicando operações contínuas como filtragens, agregações, junções entre *streams* e *windowing* temporal à medida que os eventos fluem.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

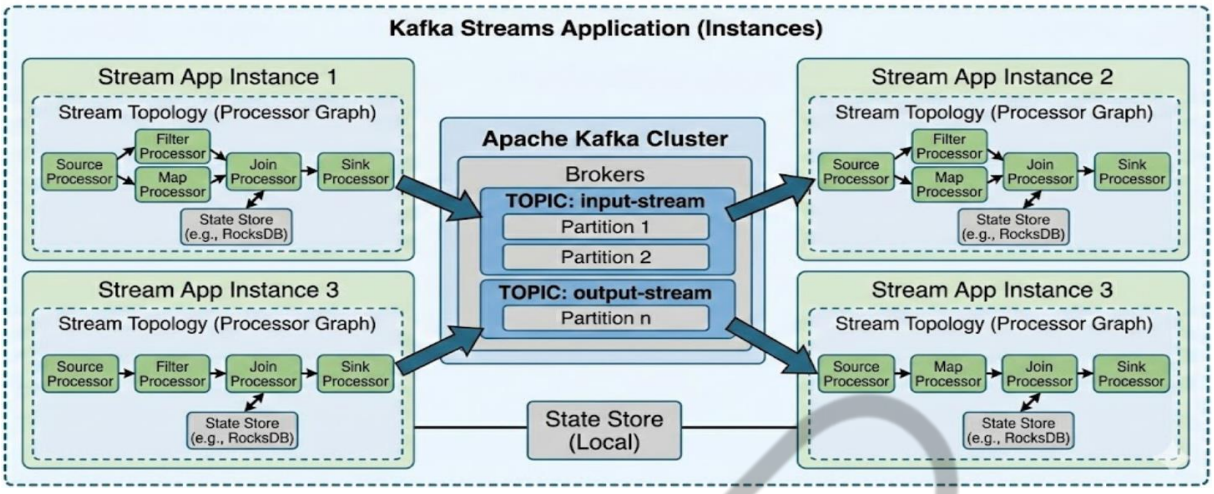


Figura 3 - Exemplificação do funcionamento Kafka Streams
Fonte: Elaborado pelo autor utilizando a ferramenta Nano Banana (2026)

Entre os conceitos fundamentais do Kafka Streams destacam-se KStream, KTable e State Store.

CONCEITOS	KSTREAM	KTABLE	STATE STORE
Tipo	Fluxo contínuo de eventos	Estado derivado por chave	Armazenamento de estado
Modelo	Event-based	State-based	Stateful
Processamento	Evento a evento	Atualiza último valor por chave	Mantém dados entre eventos
Persistência	Não persistente	Persistente (derivado)	Persistente local
Uso Principal	Eventos em tempo real	Agregação e deduplicação	Suporte a estado e janelas
Tecnologia	Kafka Topics	Topics + State Store	RocksDB ou memória
Tolerância a Falhas	Herdada do Kafka	Restauração via changelog	Replicação automática
Exemplo	Stream de transações	Saldo atual por conta	Contador por usuário

Tabela 1 – Descrições dos conceitos fundamentais do Kafka

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Fonte: elaborado pelo autor (2026)

Uma vantagem central do Kafka Streams é herdar do Kafka suas propriedades de escalabilidade e *fault tolerance*. Aplicações Kafka Streams podem ser escaladas simplesmente pela execução de múltiplas instâncias do mesmo aplicativo, permitindo que os dados sejam reparticionados e balanceados automaticamente entre elas. Em cenários de falha, outra instância assume o *partition assignment* e reprocesse os dados necessários com base no controle de *offsets* e no estado local replicado. Além disso, o Kafka Streams oferece suporte nativo à semântica de processamento *exactly-once*, integrando-se às transações do Kafka para garantir que eventos não sejam processados de forma duplicada ou perdida, mesmo em situações de falhas ou reprocessamento.

Nesse contexto, o Kafka Streams possibilita a implementação de lógicas de processamento em tempo real diretamente dentro de microsserviços ou aplicações *stand-alone*, utilizando exclusivamente a infraestrutura do Kafka. Diferentemente de frameworks de streaming externos, como Spark Streaming ou Apache Flink, o Kafka Streams não requer um cluster dedicado de processamento, pois a própria aplicação, quando executada em múltiplas instâncias, constitui o ambiente distribuído de processamento.

Para casos em que se busca uma abordagem ainda mais declarativa, o ecossistema Kafka oferece o ksqlDB, uma tecnologia construída sobre o Kafka Streams que permite definir transformações e consultas contínuas por meio de SQL. Com o ksqlDB, é possível especificar, por exemplo, filtros, agregações ou junções entre tópicos com base em chaves, e essas operações são executadas continuamente à medida que novos eventos chegam aos *streams*.

Em síntese, Kafka Streams e ksqlDB complementam o Kafka ao fornecer capacidades de processamento contínuo sobre os dados integrados. Enquanto o Kafka Connect viabiliza a extração e a carga de dados entre sistemas e tópicos, o Kafka Streams permite transformar e agregar esses dados em tempo real, formando, em conjunto, a base para pipelines ETL em tempo real completos, nos quais extração, transformação e carga ocorrem de forma contínua e em escala.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

DIMENSÃO	KAFKA CONNECT	KAFKA STREAMS	KSQLDB
Papel no Ecossistema Kafka	Integração e movimentação de dados entre sistemas externos e o Kafka	Processamento de dados em fluxo dentro do Kafka	Processamento de dados em fluxo de forma declarativa sobre Kafka
Principal Função	Extração e carga de dados (Source e Sink)	Transformação, agregação e enriquecimento de streams	Transformação e consulta contínua de streams via SQL
Linguagem / Interface	Configuração via JSON	Java ou Scala	SQL
Execução	Rodando em workers standalone ou distribuídos	Executando dentro da própria aplicação	Executado como serviço dedicado
Escalabilidade	Horizontal, via múltiplos workers	Horizontal, via múltiplas instâncias da aplicação	Horizontal, herdada do Kafka Streams
Tolerância a Falhas	Gerenciada pelo framework (offsets e rebalanceamento automático)	Herdada do Kafka (rebalanceamento e estado replicado)	Herdada do Kafka Streams
Casos de Uso Típicos	CDC, ingestão de bancos, envio para Data Lakes, DWs, motores de busca	Feature engineering em tempo real, agregações por janela, detecção de padrões	Filtros, agregações e joins em tempo real com baixo esforço

Tabela 2 – Comparação entre Kafka connect, Kafka Streams e KsqlDB
Fonte: elaborado pelo autor (2026)

Semânticas de processamento

Em pipelines de streaming, não é suficiente garantir que um evento foi apenas enviado, é fundamental compreender como o pipeline se comporta diante de falhas, retentativas e reprocessamento. Diferentemente do processamento batch, que pode ser reexecutado em janelas controladas, o streaming opera de forma contínua, lidando com redes instáveis, consumidores intermitentes e variações de carga, o que torna a semântica de entrega um fator determinante para a confiabilidade dos dados consumidos por análises e modelos.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Na semântica at-most-once, mensagens podem ser perdidas em cenários de falha, reduzindo latência e overhead, sendo adequada quando a perda de alguns eventos não compromete estatísticas globais, como métricas de telemetria. Já na semântica at-least-once, perdas são evitadas por meio de confirmações e retentativas, ao custo de possíveis duplicatas; nesse contexto, pipelines e consumidores devem ser idempotentes, utilizando identificadores únicos e mecanismos de deduplicação para evitar vieses em datasets e contagens. A semântica exactly-once (EOS) busca garantir que cada evento seja processado exatamente uma vez, mesmo sob falhas e retentativas, sendo crítica em cenários como faturamento e contabilizações sensíveis. No Kafka, a EOS é implementada por meio de transações e produtores idempotentes, implicando maior complexidade e possível impacto em latência.

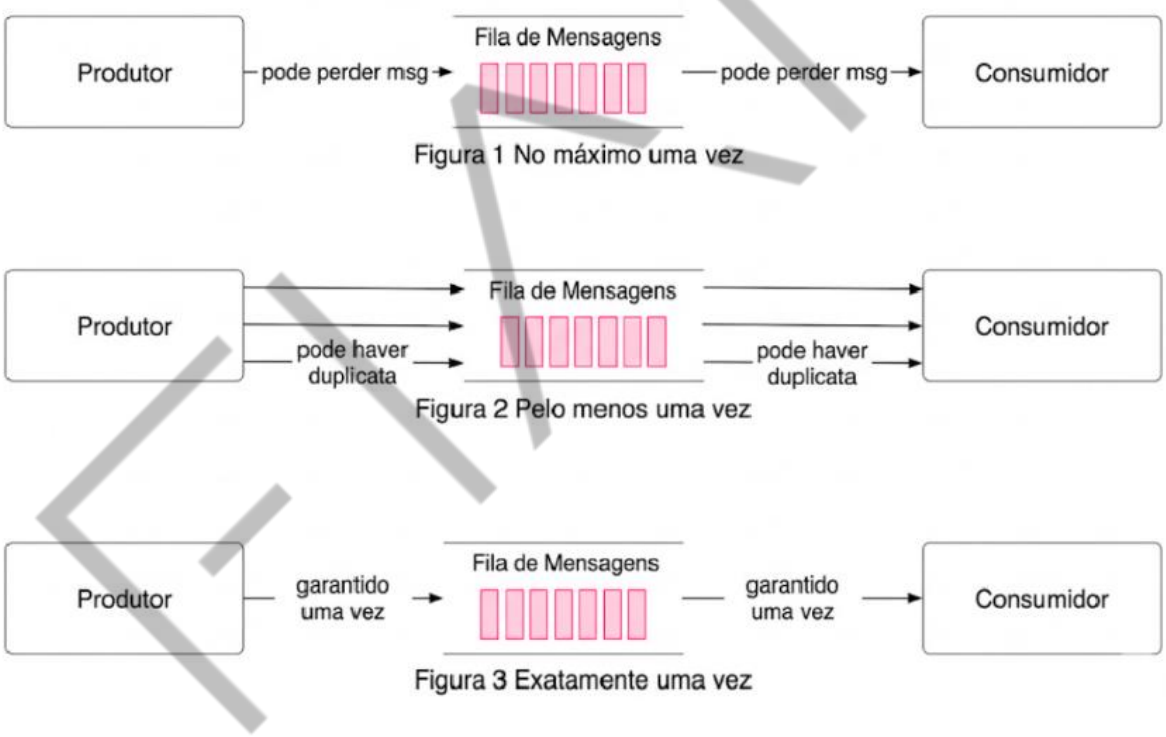


Figura 4 - Exemplificação at most once, at lest once, exactly once

Fonte: Autor, adaptador de at most once, at lest once, exactly once - Alex Xu - 2022

A escolha da semântica de processamento representa, portanto, um trade-off explícito entre latência, complexidade operacional e risco de inconsistência, devendo ser alinhada aos requisitos do caso de uso e ao impacto esperado sobre análises e modelos de Machine Learning.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Tempo em streaming: Event Time e Processing Time

Em pipelines de streaming, o tempo é parte fundamental da modelagem dos dados. Um evento pode ocorrer em um instante no mundo real e ser processado em outro momento pelo pipeline em razão de atrasos de rede, indisponibilidade de sistemas, buffers ou retentativas. Por esse motivo, distinguir event time e processing time é necessário. O event time corresponde ao instante real de ocorrência do evento, geralmente registrado no payload, enquanto o processing time representa o momento em que o evento é efetivamente processado pelo sistema.

Essa distinção impacta diretamente janelas, agregações e features temporais utilizadas por modelos de Machine Learning. Um evento ocorrido às 10:01 pode ser processado apenas às 10:10 e, caso o pipeline agregue dados exclusivamente por processing time, o comportamento do usuário pode ser interpretado de forma incorreta, afetando modelos de recomendação, detecção de fraude e previsão. Por isso, em arquiteturas baseadas em Kafka Streams e ferramentas correlatas, é recomendável adotar janelas orientadas a event time e mecanismos de tolerância a atrasos, de modo a preservar a coerência temporal entre os dados e os fenômenos que eles representam.

Boas Práticas na Integração de Dados em Tempo Real

Em arquiteturas de streaming, a adoção de um modelo orientado a eventos e desacoplado é fundamental. Ao estruturar a integração no padrão *publish/subscribe*, produtores e consumidores não mantêm dependência direta, o que aumenta flexibilidade, escalabilidade e resiliência. Nesse modelo, serviços podem evoluir de forma independente e processar eventos em ritmos distintos, enquanto o Kafka atua como intermediário confiável, retendo eventos quando consumidores estão indisponíveis e permitindo a inclusão de novos consumidores sem impacto nos produtores.

Em integrações com bancos de dados transacionais, como sistemas de vendas, ERP ou CRM, recomenda-se o uso de Change Data Capture em vez de consultas periódicas completas. O CDC permite capturar apenas as alterações ocorridas, inserções, atualizações e deleções, com impacto mínimo no desempenho

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

da base de origem. Ferramentas como Debezium, integradas ao Kafka Connect, fornecem conectores prontos para diversos SGBDs, simplificando a replicação contínua de dados. A escolha do método de CDC, seja baseada em logs, *triggers* ou outras abordagens, deve equilibrar o frescor dos dados e carga no sistema de origem, conforme os requisitos do caso de uso.

Para integração entre sistemas externos e o Kafka, é preferível utilizar Kafka Connect em vez de implementar soluções customizadas. Conectores oficiais ou da comunidade encapsulam boas práticas consolidadas, incluindo tratamento de falhas, reconexões, controle de offsets e otimizações de desempenho, além de oferecerem monitoramento centralizado e isolamento de problemas. Ao utilizar Kafka Connect, reduz-se significativamente o código *boilerplate* e risco de erros, permitindo que a equipe foque na lógica de negócio da integração, em vez de detalhes de conectividade.

Em ambientes de dados distribuídos em tempo real, governança e evolução de schemas tornam-se aspectos críticos. A definição de contratos claros para eventos, utilizando formatos estruturados como JSON com esquema ou, preferencialmente, Avro ou Protobuf integrados a um Schema Registry, permite versionar esquemas e validar compatibilidade entre produtores e consumidores. Esse controle evita que alterações estruturais quebrem pipelines ou causem inconsistências semânticas, além de facilitar o consumo confiável dos dados por múltiplos times, prevenindo a degradação do ecossistema em um *data swamp*.

O monitoramento contínuo do pipeline é igualmente essencial. Métricas como latência ponta a ponta, throughput e *consumer lag* permitem identificar gargalos e antecipar problemas de capacidade. Ferramentas como Kafka Manager, Confluent Control Center e stacks baseados em Grafana e Prometheus possibilitam acompanhar o quão atrasados consumidores estão em relação ao final dos tópicos, indicando a necessidade de escalonamento horizontal ou otimização. Em casos de uso com requisitos estritos de tempo real, o *lag* deve permanecer próximo de zero ou dentro de SLAs claramente definidos, sendo recomendável a realização de testes de carga para calibrar número de partições e instâncias de consumidores e conectores.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Por fim, o Kafka oferece mecanismos de autenticação e autorização, como SASL/SSL e ACLs, que permitem restringir o acesso a tópicos apenas a sistemas autorizados. A criptografia de dados em trânsito, via TLS, e em repouso, nos discos dos brokers, contribui para a proteção de informações sensíveis. Manter o cluster atualizado com correções de segurança e, quando necessário, adotar plataformas gerenciadas com certificações de conformidade reforça a postura de segurança do ambiente. Independentemente das particularidades de cada caso de uso, os princípios de baixo acoplamento, governança rigorosa e foco na qualidade dos dados permanecem aplicáveis de forma geral.

Cheatsheet de Comandos CLI

Abaixo estão os comandos nativos que você executará dentro do container Kafka (ou de sua máquina local, se tiver o Kafka instalado).

1. Gestão de Tópicos (kafka-topics.sh) Cria, lista e descreve a estrutura dos dados:

```
# Cria um tópico 'sensores_v1' com 3 partições e replicação
                        fator 1

 kafka-topics.sh --create --topic sensores_v1 --bootstrap-
server localhost:9092 --partitions 3 --replication-factor 1
```

Código-fonte 1 – Criar um novo tópico

Fonte: Modificado pelo autor com base na documentação do Kafka (2026)

Listar tópicos existentes:

```
kafka-topics.sh --list --bootstrap-server localhost:9092
```

Código-fonte 2 – Listar tópicos existentes

Fonte: Modificado pelo autor com base na documentação do Kafka (2026)

Descrever detalhes (Debug): Útil para ver se as partições estão distribuídas corretamente:

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

```
kafka-topics.sh --describe --topic sensores_v1 --bootstrap-
server localhost:9092
```

Código-fonte 3 – Listar tópicos existentes

Fonte: Modificado pelo autor com base na documentação do Kafka (2026)

2. Produção e Consumo no Terminal (kafka-console-*) Ferramentas rápidas para injetar ou ler dados manualmente.

Consumir dados em tempo real (O comando mais usado!): Use isso para verificar se os dados estão chegando:

```
#Lê desde o início do histórico (--from-beginning) kafka-
console-consumer.sh --topic sensores_v1 --from-beginning --
bootstrap-server localhost:9092
```

Código-fonte 4 – Leitura do histórico

Fonte: Modificado pelo autor com base na documentação do Kafka (2026)

Produzir dados manualmente: ótimo para testar como seu modelo reage a inputs específicos:

```
kafka-console-producer.sh --topic sensores_v1 --bootstrap-
server localhost:9092
# Após dar enter, digite suas mensagens:
> {"id": 1, "temp": 25.5}
> {"id": 2, "temp": 88.0}
```

Código-fonte 5 – Produção de dados

Fonte: Modificado pelo autor com base na documentação do Kafka (2026)

3. Monitoramento de Grupos (kafka-consumer-groups.sh) muito utilizado para MLOps. Permite ver o Lag (atraso). Se o Lag estiver alto, seu modelo está demorando muito para processar os dados e tomando decisões com base em informações defasadas.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Verificar o Lag:

```
kafka-consumer-groups.sh --bootstrap-server localhost:9092 --
describe --group etl_group_v1
```

Código-fonte 6 – Monitorar o atraso

Fonte: Modificado pelo autor com base na documentação do Kafka (2026)

Agora fica um desafio para você, que tal criar um projeto usando a biblioteca padrão kafka-python? Dicas:

Crie o produtor producer.py;

Crie o consumidor consumer.py.

Como testar: rode o consumer em um terminal: python consumer.py (ele ficará travado esperando). Em outro terminal, rode o producer: python producer.py. Veja a mensagem aparecer instantaneamente no primeiro terminal.

Integração em Nuvem vs On-Premises

Ao implementar soluções com Apache Kafka, uma decisão arquitetural central consiste em escolher entre executar a infraestrutura localmente (*on-premises*) ou utilizar serviços gerenciados em nuvem. Ambas as abordagens apresentam vantagens e limitações, sendo comum a adoção de arquiteturas híbridas. A escolha impacta diretamente controle operacional, custo, escalabilidade e conformidade regulatória.

No modelo on-premises (auto-hospedado), a organização instala e gerencia o Kafka em seus próprios servidores, seja em datacenters locais ou em máquinas virtuais autogerenciadas. A principal vantagem é o controle total sobre configuração, *tunings*, versões e políticas de segurança, permitindo customizar parâmetros de *brokers*, integrar livremente com sistemas legados e atender requisitos regulatórios ou de dados sensíveis. Em contrapartida, esse modelo exige uma equipe especializada para administrar o cluster, incluindo instalação, *upgrades*, monitoramento, *tuning* de desempenho e recuperação de falhas. Também é necessário provisionar adequadamente CPU, memória, disco, preferencialmente SSDs, e rede, além de planejar a escalabilidade manual por meio da adição de *nodes*

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

e reparticionamento de tópicos. Embora o Kafka seja projetado para tolerância a falhas e escalabilidade horizontal, no ambiente on-premises cabe à equipe garantir esses mecanismos, configurando replicação de partições, distribuição de *brokers* entre racks, múltiplos *ZooKeepers* para *quorum*, backups e *mirroring* (MirrorMaker) entre datacenters. Assim, o on-premises oferece máxima flexibilidade ao custo de maior responsabilidade operacional.

No ambiente em nuvem com serviços gerenciados, provedores e empresas especializadas oferecem Kafka como serviço ou plataformas compatíveis, como Amazon MSK, Azure Event Hubs (com interface Kafka), Confluent Cloud e IBM Event Streams. Nessas soluções, grande parte da administração é abstraída do usuário, incluindo provisionamento de *brokers*, substituição automática em falhas, aplicação de *patches*, atualizações, escalonamento de partições e monitoramento básico. Muitos serviços oferecem recursos adicionais, como *tiered storage*, armazenamento elástico e integração nativa com outros serviços de nuvem. A principal vantagem é a agilidade, permitindo criar clusters funcionais em minutos e liberar a equipe para focar no desenvolvimento de produtores, consumidores e tópicos, em vez da manutenção da infraestrutura.

Entretanto, serviços gerenciados implicam custos potencialmente mais elevados por volume de dados e alguma perda de flexibilidade. Podem existir restrições de versão, configuração ou ausência de acesso *root* para customizações específicas. Aspectos de segurança e conformidade também devem ser avaliados, pois, embora os provedores ofereçam certificações como SOC, ISO e GDPR, algumas organizações exigem isolamento completo de determinados dados.

Um modelo híbrido é frequentemente adotado, combinando Kafka on-premises para coleta de dados de sistemas internos com replicação em tempo real para clusters em nuvem que alimentam *data lakes* e plataformas analíticas. Nesse cenário, é necessário considerar latência de rede e custos de transferência de dados entre ambientes. Além disso, o Kafka pode ser executado tanto on-premises quanto na nuvem em ambientes de contêineres e orquestração, como Kubernetes, utilizando operadores que simplificam a gestão e o *deploy*.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

A decisão entre on-premises, nuvem ou híbrido deve considerar fatores como expertise da equipe, urgência de entrega, escala esperada, orçamento, requisitos de compliance e estratégia de TI de longo prazo. Independentemente do ambiente escolhido, os conceitos e componentes do ecossistema Kafka — como Kafka, Kafka Connect e Kafka Streams — permanecem os mesmos.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

MERCADO, CASES E TENDÊNCIAS

No mercado corporativo, Kafka é amplamente adotado por empresas de tecnologia, finanças, varejo, logística e mídia para resolver problemas de escala, latência e desacoplamento entre sistemas. Empresas como LinkedIn, Netflix, Uber, Airbnb e grandes bancos utilizam Kafka como backbone de integração, permitindo ingestão contínua de eventos, sincronização entre microsserviços e alimentação de data lakes e feature stores em tempo real. No setor financeiro, Kafka é utilizado para processamento de transações, detecção de fraude e monitoramento de risco, onde latência e consistência são críticas. No varejo e e-commerce, viabiliza recomendações em tempo real, precificação dinâmica e monitoramento de jornadas de compra. Já em plataformas digitais e de mídia, sustenta coleta de eventos de navegação, personalização de conteúdo e análise de audiência em escala.

Indicação de artigos e vídeos:

“O que é Kafka” – Alura - Youtube:

https://www.youtube.com/watch?v=G6Tcy7hNdA8

Indicação de livros:

“Designing Event-Driven Systems” – Ben Stopford: apresenta os princípios, padrões e decisões arquiteturais por trás de sistemas orientados a eventos, explorando como o streaming de dados sustenta arquiteturas modernas e escaláveis, com forte relação ao uso do Apache Kafka em ambientes corporativos.

“Fundamentals of Data Engineering” – Joe Reis, Matt Housley: apresenta uma visão moderna da engenharia de dados, cobrindo ingestão, pipelines batch e streaming, governança, confiabilidade e escolha tecnológica. O livro contextualiza o uso de Kafka dentro de arquiteturas reais orientadas a dados e IA.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você aprendeu como a integração de dados em tempo real é fundamental para arquiteturas modernas de dados e Inteligência Artificial, especialmente em cenários que exigem baixa latência e resposta imediata a eventos. A transição da Arquitetura Lambda para a Arquitetura Kappa evidenciou o streaming como eixo central do processamento, simplificando arquiteturas e reduzindo redundâncias.

Você compreendeu o papel do Apache Kafka como uma plataforma distribuída de streaming de eventos, baseada em log distribuído, tópicos e partições, capaz de desacoplar sistemas e sustentar pipelines escaláveis e tolerantes a falhas. Foram explorados seus principais componentes: Kafka Connect para integração e CDC, Kafka Streams para processamento em fluxo com gerenciamento de estado e semântica exactly-once, e ksqlDB para processamento declarativo via SQL.

O ecossistema do Kafka foi explorado a partir de seus principais componentes. O Kafka Connect foi apresentado como a solução padrão para integração entre sistemas externos e o Kafka, suportando cenários de ingestão e entrega de dados, incluindo Change Data Capture com ferramentas como Debezium. O Kafka Streams foi estudado como a camada de processamento em fluxo, permitindo transformações, agregações, junções, janelas temporais e gerenciamento de estado por meio de KStreams, KTables e State Stores, com suporte a semântica exactly-once. Complementarmente, o ksqlDB mostrou como essas capacidades podem ser acessadas de forma declarativa via SQL, ampliando a autonomia de analistas e cientistas de dados.

Você também analisou conceitos críticos para pipelines streaming confiáveis, como semânticas de processamento (at-most-once, at-least-once e exactly-once), a distinção entre event time e processing time, e o impacto dessas escolhas na qualidade dos dados. Aspectos de governança, como versionamento de schemas com Schema Registry, foram discutidos como fundamentais para evitar que mudanças estruturais quebrem pipelines ou comprometam análises.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

Por fim, a aula abordou boas práticas de engenharia de dados em streaming, incluindo arquitetura orientada a eventos, uso de CDC, monitoramento de lag e latência, segurança, controle de acesso e estratégias de implantação do Kafka em ambientes on-premises, nuvem ou híbridos. Você viu como essas decisões arquiteturais influenciam diretamente custo, escalabilidade, governança e tempo de entrega, reforçando que a integração de dados em tempo real é um elemento estratégico para soluções modernas de dados e Inteligência Artificial.

Mapa Mental



Figura 4 - Mapa mental com o conteúdo abordado
Fonte: Elaborado pelo autor utilizando a ferramenta Canva IA (2026)

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

REFERÊNCIAS

ALICE TECH. ***Descomplicando o Apache Kafka***. Medium, 2019. Disponível em: https://medium.com/alice-tech/descomplicando-o-apache-kafka-5776d96423d. Acesso em: 24 mar. 2026.

AMAZON WEB SERVICES. **O que é Apache Kafka?** 2024. Disponível em: https://aws.amazon.com/pt/what-is/apache-kafka/. Acesso em: 24 mar. 2026.

AMAZON WEB SERVICES. **What is Data Integration?** – Explanation of Data Integration Technologies. 2025. Disponível em: https://aws.amazon.com/pt/what-is/data-integration/. Acesso em: 24 mar. 2026.

APACHE SOFTWARE FOUNDATION. ***Apache Kafka – Introduction***. 2024. Disponível em: https://kafka.apache.org/41/getting-started/introduction/. Acesso em: 24 mar. 2026.

CONFLUENT. **Kafka Connect Data Transformation and Schema Registry**. Confluent Blog. Disponível em: https://www.confluent.io/blog/kafka-connect-data-transformation-schema/. Acesso em: 24 mar. 2026.

CONFLUENT. **Kafka Connect Design**. Disponível em: https://docs.confluent.io/platform/current/connect/design.html. Acesso em: 24 mar. 2026.

ESTUARY. **Kafka Event-Driven Architecture**. Disponível em: https://estuary.dev/blog/kafka-event-driven-architecture/. Acesso em: 24 mar. 2026.

GULATI, Jayita. **Creating a Data Science Pipeline for Real-Time Analytics Using Apache Kafka and Spark**. KDnuggets, 2025. Disponível em: https://www.kdnuggets.com/creating-a-data-science-pipeline-for-real-time-analytics-using-apache-kafka-and-spark. Acesso em: 24 mar. 2026.

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

INSTACLUSTR. **Building a Data Pipeline with Apache Kafka & Kafka Connect**. Disponível em: https://www.instaclustr.com/education/apache-kafka/building-a-data-pipeline-with-apache-kafka-kafka-connect-the-basics-and-best-practices/. Acesso em: 24 mar. 2026.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com

PALAVRAS-CHAVE

Streaming. Escalabilidade. Confiabilidade.

FIAP

PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com



PDF exclusivo para Cleivin Lauermann - rm376917
cleivin.lauermann@gmail.com